Metody Inteligencji Obliczeniowej

Sprawozdanie 1

Wydział Fizyki i Informatyki Stosowanej

Mikhail Shupliakou

15.03.2026

Spis treści

1 Wprowadzenie **3**

2 Zadanie 1 **3**
2.1 Cel i opis zadania 3
2.2 Generacja danych 3
2.3 Podział na zbiory i Trening klasyfikatora 5
2.4 Wizualizacja i ewaluacja 6
2.4.1 Dla 5: 6
2.4.2 Dla 10: 7
2.4.3 Dla 20: 7
2.4.4 Dla 100: 9
2.5 Wnioski 10

3 Zadanie 2 **10**
3.1 Przygotowanie zbioru danych 10
3.2 Budowa warstwy perceptronów 11
3.3 Wyniki 11
3.3.1 Wnioski 11

4 Zadanie 3 **12**
4.1 80% - dane do testowania 13
4.2 55% - dane do testowania 13
4.3 10% - dane do testowania 14

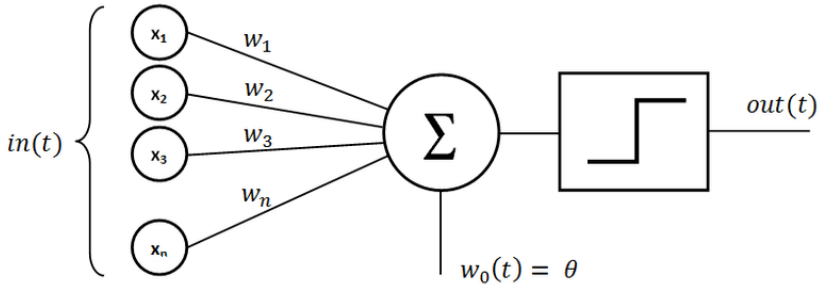
5 Zadanie 4 **15**
5.1 Wnioski 16

6 Spis Literatury i odpowiedne linki **17**

Klasyfikacja z Użyciem Sztucznych Neuronów

1 Wprowadzenie

Perceptron - prosta sztuczna sieć neuronowa, wprowadzona przez Franka Rosenblatta w 1958 roku, dla której podał metodę jej uczenia, czyli stopniowego dobierania wag i biasów (progów) tak, by wyniki podawane na wyjściu sieci były coraz bardziej zgodne z wartościami oczekiwanymi. W swojej podstawowej formie (perceptron prosty) składa się z jednego sztucznego neuronu McCullocha-Pittsa ze skokową funkcją aktywacji i regulowanymi wagami i progiem. Dalszy rozwój doprowadził do konstrukcji perceptronów wielowarstwowych, w tym perceptronów ze sprzężeniem zwrotnym, z różnymi funkcjami aktywacji, mających znacznie większe możliwości obliczeniowe; istotne było tu zastosowanie ciągłych funkcji aktywacji i odkrycie nowych metod uczenia sieci neuronowych, by przezwyciężyć ograniczenia perceptronu Rosenblatta, który mógł uczyć się jedynie problemów liniowo separowalnych.



Rysunek 1: Perceptron złożony z jednego neuronu McCullocha-Pittsa: ma n wejść (dendrytów) i 1 wyjście (akson); służy do wykrywania pojedynczej cechy w danych; każda dana jest zapisana w postaci wektora mającego n składowych.

2 Zadanie 1

2.1 Cel i opis zadania

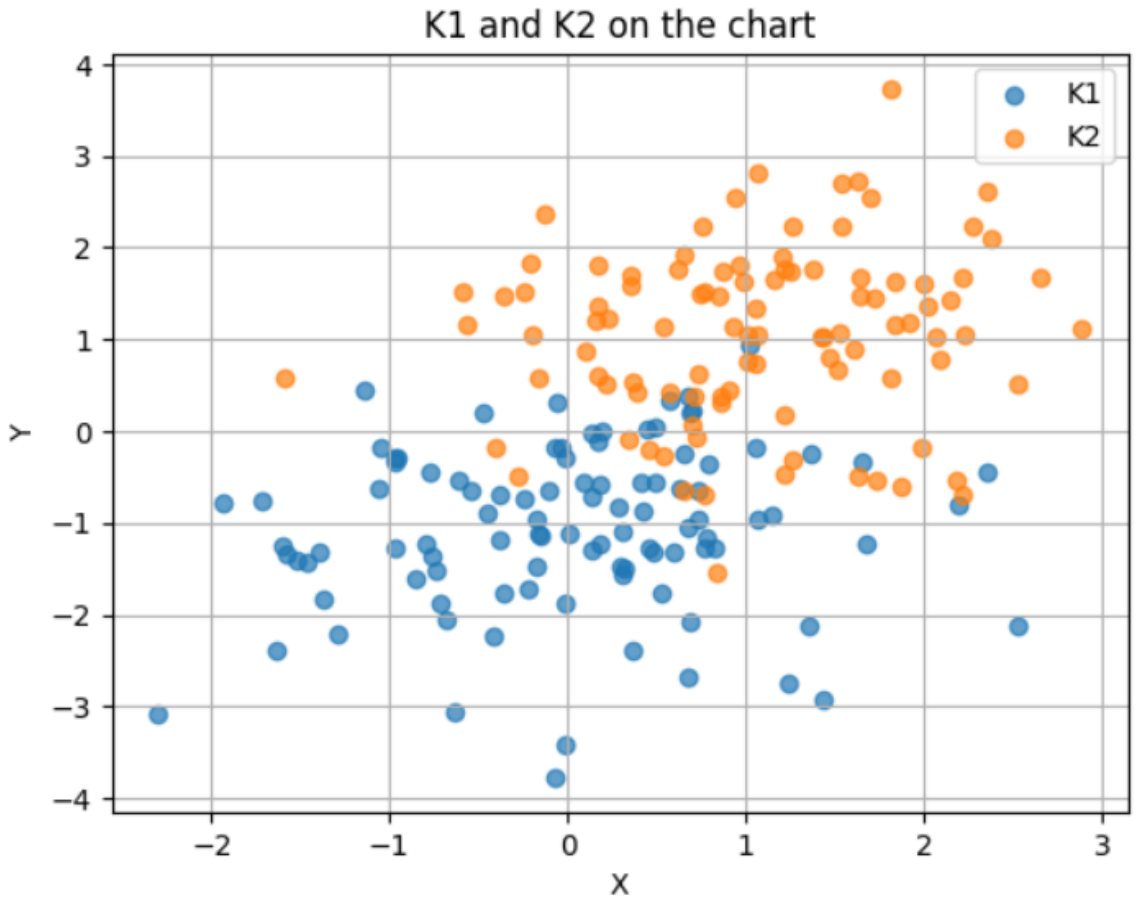
Głównym celem zadania jest zbadanie wpływu rozmiaru zbioru uczącego na jakość, stabilność i zdolność uogólniania klasyfikatora liniowego. Realizacja zadania składa się z następujących kroków:

2.2 Generacja danych

Utworzyłem zbiór 200 punktów w przestrzeni dwuwymiarowej.

```
K1 = np.random.multivariate_normal([0, -1], [[1, 0], [0, 1]], size=100)
K2 = np.random.multivariate_normal([1, 1], [[1, 0], [0, 1]], size=100)
```

Punkty są równo podzielone na dwie klasy, losowane z rozkładów normalnych: klasa K_1 ze średnią $\mu_1 = [0, -1]$ i klasa K_2 ze średnią $\mu_2 = [1, 1]$. Wariancja dla obu klas wynosi 1.



Rysunek 2: Gotowy zbiór

2.3 Podział na zbiory i Trening klasyfikatora

Przygotowałem warianty zbiorów uczących o różnej liczebności: 5, 10, 20 oraz 100 punktów.

W tym pomogła mi funkcja `teachNeuron(K1learn, K2learn)`:

```
def teachNeuron(K1learn, K2learn):
    # przygotowujemy wzorcowe dane uczące. X to współrzędne, y to klasa, do
    ↪  której należą
    X = np.vstack((K1learn, K2learn))
    y = [0]*len(K1learn)+[1]*len(K1learn)

    # przygotowujemy perceptron.
    neuron = Perceptron(tol=1e-3, max_iter = 20)

    # uczymy neuron przez wskazaną liczbę epok lub do zatrzymania się uczenia
    neuron.fit(X, y)

    # some code was left behind...

    # możemy sprawdzić jak udane było uczenie:
    print("Jakość nauki neuron.score(X, y): ",neuron.score(X, y))
    print("Wagi neuron.coef_[0]): ",neuron.coef_[0])
    print("Bias: ",neuron.intercept_[0])
    return neuron.coef_[0], neuron.intercept_[0]
```

Trenowanie perceptrona na 5 punktach:

```
wagi, bias = teachNeuron(K1[0:5], K2[0:5])
```

Analogicznie, funkcja została użyta na zbiorach o liczebności 10, 20 i 100 punktów.

Funkcja `teachNeuron(K1learn, K2learn)` nie tylko dzieli zbiory na odpowiednie podzbiory, ale również uczy neuron klasyfikowania.

Trening klasyfikatora polega na wyznaczeniu równania prostej (hiperpłaszczyzny decyzyjnej), która najlepiej separuje klasy K_1 i K_2 dla każdego z przygotowanych wariantów zbioru uczącego.

Po trenowaniu perceptron przechowuje swoje parametry:

- `neuron.coef_[0]` – wagi przypisane do cech. Jest to tablica postaci $[w_1, w_2]$ (współczynniki stojące przy x_1 i x_2).
- `neuron.intercept_[0]` – wyraz wolny (bias), oznaczmy go jako b.

Równanie powierzchni decyzyjnej (w przypadku dwóch cech jest to prosta) ma postać:

$$w_1x_1 + w_2x_2 + b = 0.$$

Aby narysować tę prostą, należy wyrazić x_2 przez x_1:

$$x_2 = -\frac{w_1x_1 + b}{w_2}.$$

Przyjmujemy przy tym, że $w_2 \neq 0$.

```
#wewnątrz funkcji teachNeuron(K1learn, K2learn)
x1=np.linspace(-5, 5, 100)
x2 = -(1./neuron.coef_[0][1])*(neuron.coef_[0][0]*x1 + neuron.intercept_[0])
```

```
plt.plot(x1, x2, '-r')
```

2.4 Wizualizacja i ewaluacja

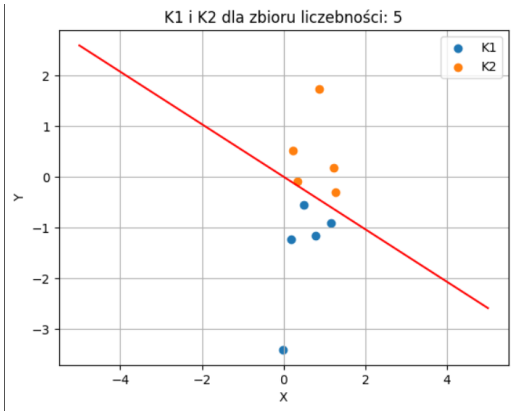
Do oceny wyników została napisana funkcja licząca **Accuracy** i tworząca **Confusion Matrix** `buildConfusionMatrix(k1, k2, wagi, bias)`:

```
y_true = [0] * len(k1) + [1] * len(k2)
X_all = np.vstack((k1, k2))

#K1 znajduje się pod prostą, K2 - nad
scores = np.dot(X_all, wagi) + bias
y_pred = (scores > 0).astype(int)

from sklearn import metrics
cm = metrics.confusion_matrix(y_true, y_pred)

cm_display = metrics.ConfusionMatrixDisplay(confusion_matrix=cm,
↪  display_labels=['K1', 'K2'])
cm_display.plot(cmap='Blues')
plt.title("Confusion Matrix")
plt.show()

accuracy = metrics.accuracy_score(y_true, y_pred)
print(f"Accuracy: {accuracy:.2%}")
```

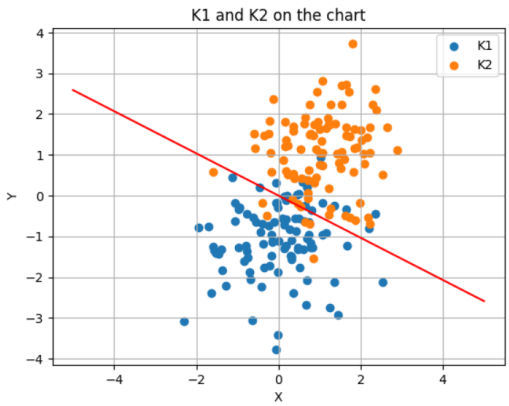
Otrzymane wyniki:

2.4.1 Dla 5:

```
Jakosc nauki neuron.score(X, y):  1.0
Wagi neuron.coef_[0]):  [1.03047995 1.99056967]
Bias:  0.0
```

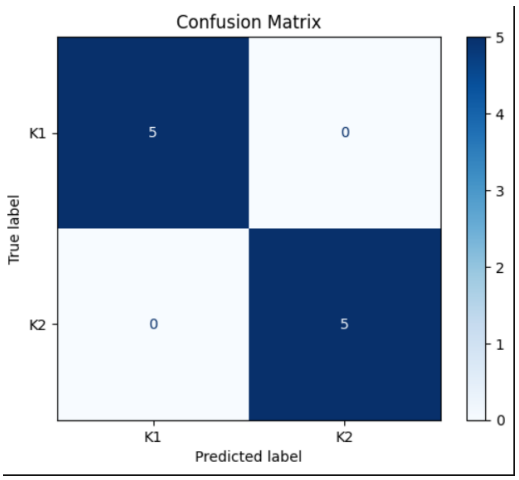


Rysunek 3: Liczebność: 5. Prosta na wykresie danych do trenowania.

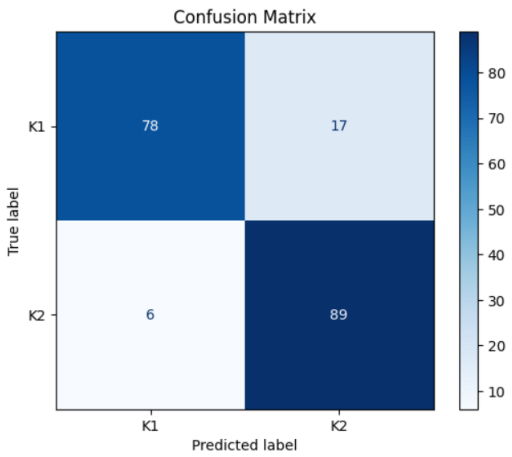


Rysunek 4: Liczebność: 5. Prosta na wykresie całego zbioru.

```
Dla danych do nauki
Accuracy: 100.00%

Dla danych testowych
Accuracy: 87.89%
```



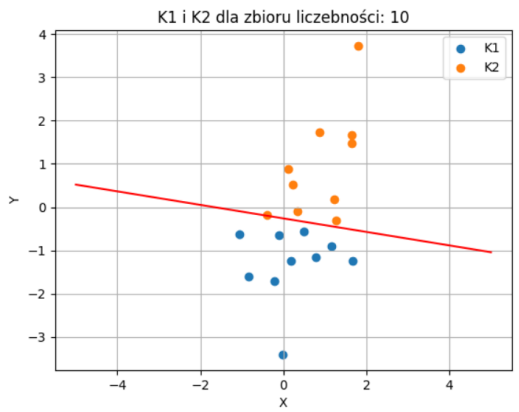
Rysunek 5: Liczebność: 5. Confusion Matrix na danych do trenowania.



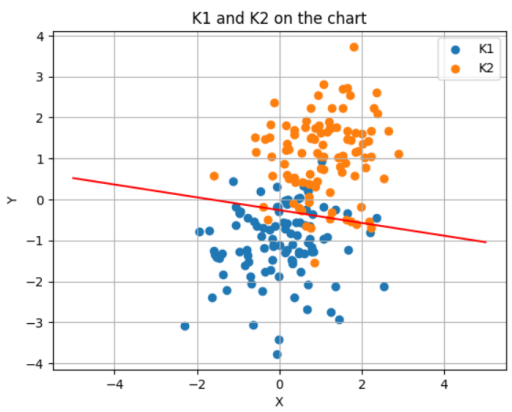
Rysunek 6: Liczebność: 5. Confusion Matrix na całym zbiorze.

2.4.2 Dla 10:

```
Jakosc nauki neuron.score(X, y):  1.0
Wagi neuron.coef_[0]):  [0.60137147 3.84811262]
Bias:  1.0
```



Rysunek 7: Liczebność: 10. Prosta na wykresie danych do trenowania.



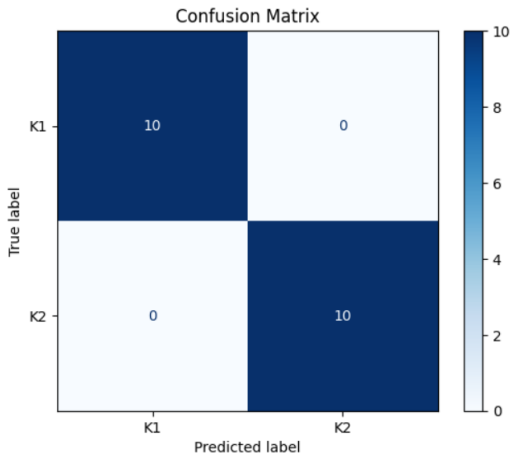
Rysunek 8: Liczebność: 10. Prosta na wykresie całego zbioru.

```
Dla danych do nauki
Accuracy: 100.00%

Dla danych testowych
Accuracy: 84.44%
```

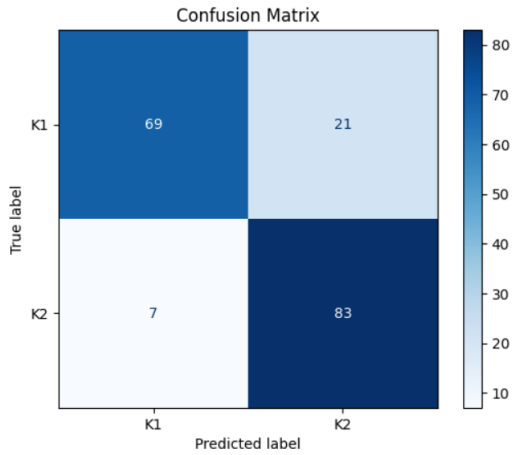
2.4.3 Dla 20:

```
Jako nauki neuron.score(X, y):  0.775
Wagi neuron.coef_[0]):  [0.07088781 4.11809474]
Bias:  -1.0
```

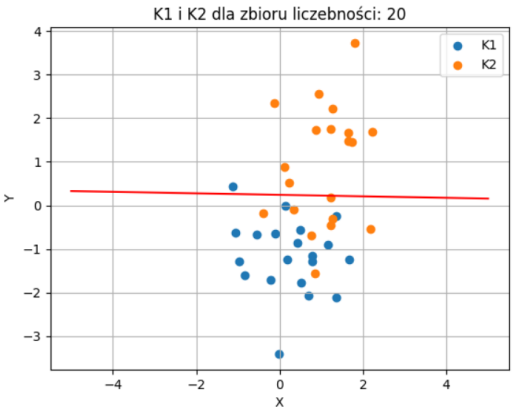
```
Dla danych do nauki
Accuracy:  77.50%

Dla danych testowych
Accuracy: 90.62%
```



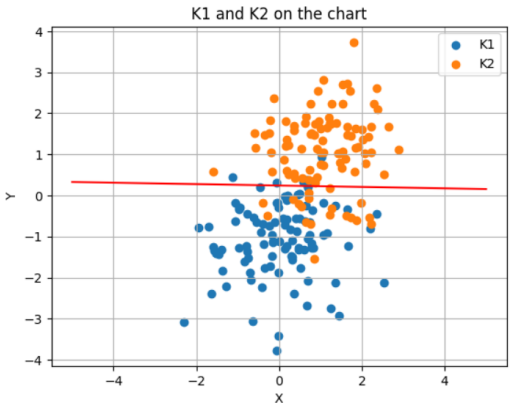
Rysunek 9: Liczebność: 10. Confusion Matrix na danych do trenowania.



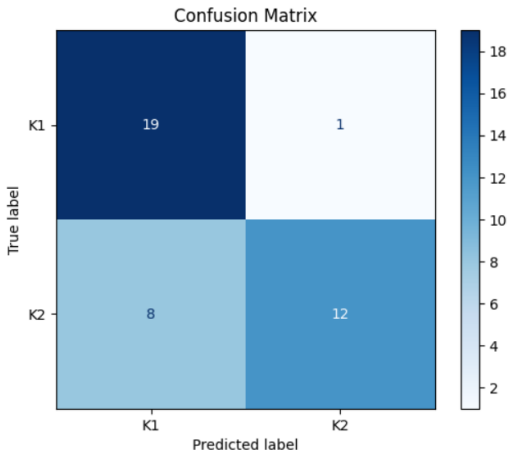
Rysunek 10: Liczebność: 10. Confusion Matrix na całym zbiorze.



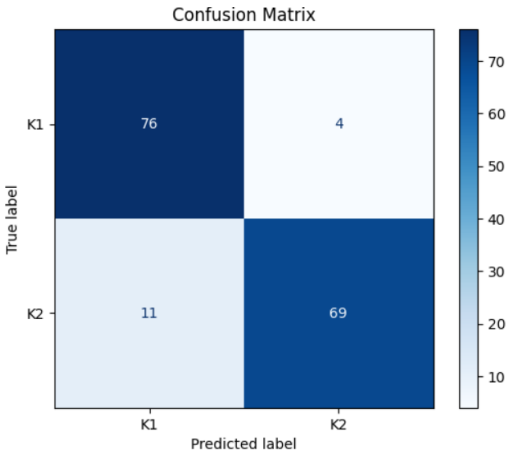
Rysunek 11: Liczebność: 20. Prosta na wykresie danych do trenowania.



Rysunek 12: Liczebność: 20. Prosta na wykresie całego zbioru.



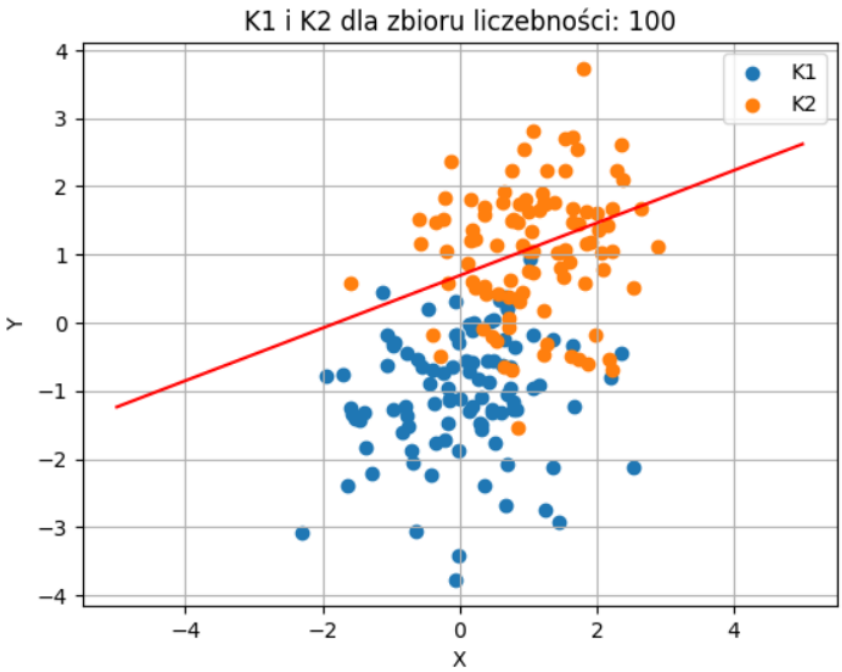
Rysunek 13: Liczebność: 20. Confusion Matrix na danych do trenowania.



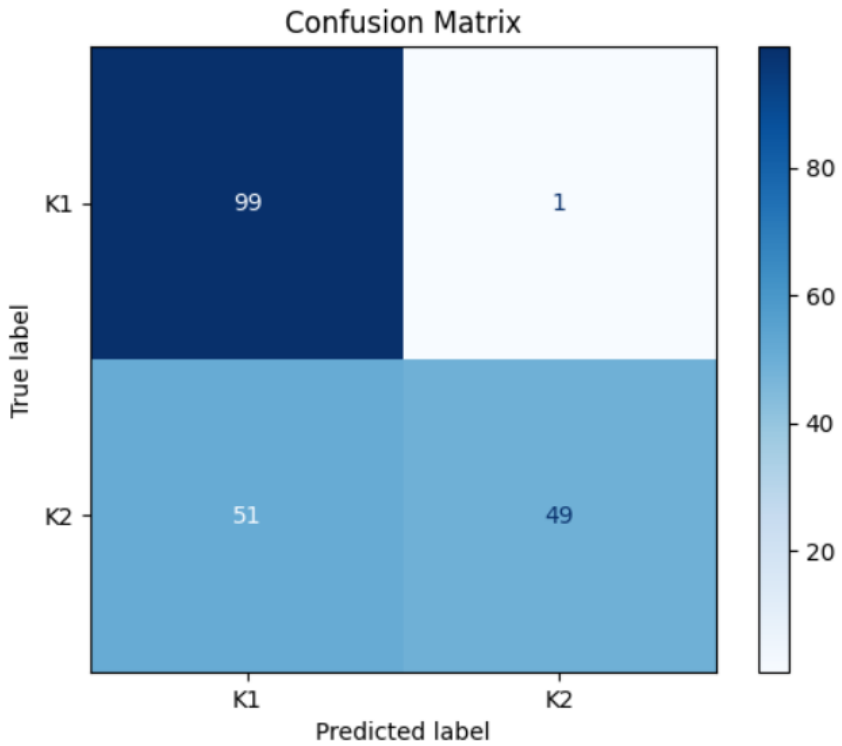
Rysunek 14: Liczebność: 20. Confusion Matrix na całym zbiorze.

2.4.4 Dla 100:

```
Jakosc nauki neuron.score(X, y):  0.74
Wagi neuron.coef_[0]):  [-0.55755812  1.44465897]
Bias:  -1.0
```



Rysunek 15: Liczebność: 100. Prosta na wykresie całego zbioru.



Rysunek 16: Liczebność: 1000. Confusion Matrix na całym zbiorze.

```
Dla danych do nauki
Accuracy:  74.00%
```

2.5 Wnioski

Na podstawie przeprowadzonych eksperymentów klasyfikacji przy użyciu perceptronu można wyciągnąć następujące wnioski:

- **Zjawisko przeuczenia (overfitting) dla małych prób:** W przypadku najmniejszych zbiorów uczących ($N = 5$ oraz $N = 10$) model osiągnął 100% skuteczności na danych treningowych. Świadczy to o tym, że klasyfikator idealnie dopasował się do nielicznych próbek, jednak wyznaczona prosta decyzyjna nie była optymalna dla całej populacji (niższa celność na danych testowych).
- **Poprawa uogólniania:** Przy zwiększeniu próby uczącej do $N = 20$, dokładność na zbiorze uczącym spadła (do 77.5%), co jest zjawiskiem naturalnym, jednak dokładność na danych testowych wzrosła do ponad 90%. Model zaczął lepiej "rozumieć"ogólny trend danych.
- **Brak separowalności liniowej a zbieżność modelu:** Zbiory wygenerowane z rozkładów normalnych z wariancją równą 1 wykazują znaczne nakładanie się na siebie, przez co klasy K_1 i K_2 nie są liniowo separowalne.
- **Niestabilność perceptronu dla $N = 100$:** Skutkiem braku separowalności jest wynik dla $N = 100$, gdzie ogólna skuteczność wyniosła zaledwie 74%. Klasyczny algorytm perceptronu nie osiąga zbieżności dla danych nieseparowalnych liniowo. Przy narzuconym ograniczeniu liczby iteracji (`max_iter=20`), algorytm zatrzymał się w punkcie suboptymalnym, co doprowadziło do silnego faworyzowania jednej z klas (błędna klasyfikacja 51 punktów z klasy K_2).
- **Podsumowanie:** Zwiększenie rozmiaru zbioru uczącego pomaga zredukować losowość i poprawić uogólnianie; jednak do rozwiązywania problemów z nakładającymi się klasami (dane liniowo nieseparowalne) prosty perceptron jest niewystarczający.

3 Zadanie 2

Celem ćwiczenia jest budowa i ocena modelu klasyfikatora opartego na jednowarstwowej sieci perceptronów dla klasycznego zbioru danych Iris. Zakres prac obejmuje pobranie danych, ich podział na zbiór uczący (80%) i testowy (20%), a następnie wytrenowanie modelu, na przykład przy użyciu klasy `Perceptron` z biblioteki `scikit-learn`.

3.1 Przygotowanie zbioru danych

Na początek pobrałem zbiór https://archive.ics.uci.edu/ml/datasets/iris. Zrobiłem to za pomocą funkcji `sklearn.datasets.load_iris()`. Dokonałem podziału na dane uczące i testowe.

```
dataset = sklearn.datasets.load_iris()

X=dataset.data
y = dataset.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↪  random_state=42)
```

3.2 Budowa warstwy perceptronów

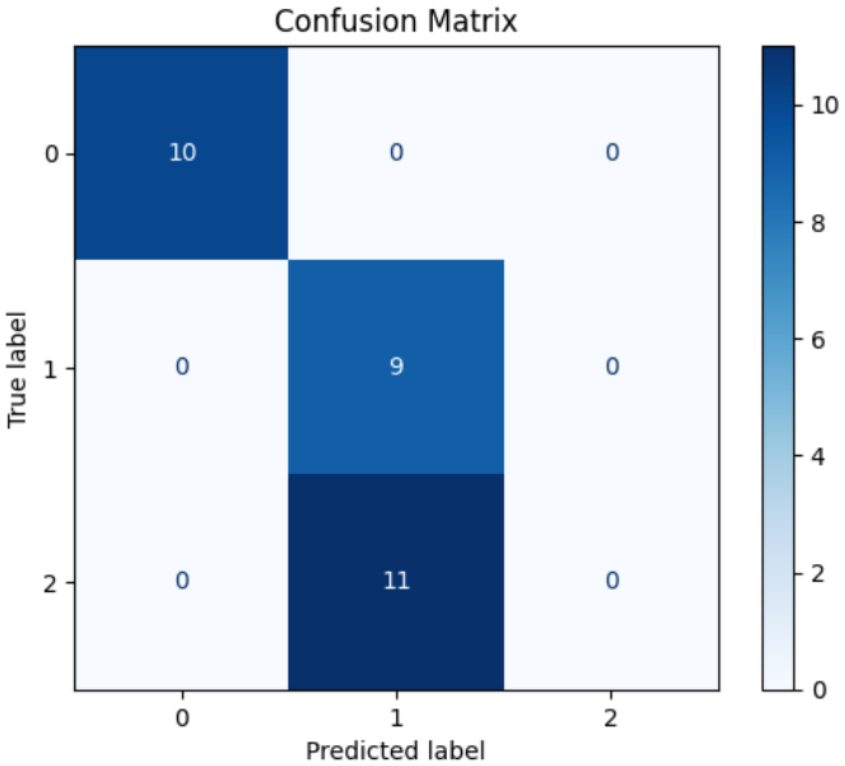
Używając funkcji `sklearn.linear_model.Perceptron` zbudowałem sieć złożoną z pojedynczej warstwy perceptronów.

```
perceptron_layer = Perceptron(max_iter=1000, tol=1e-3, random_state=42)
perceptron_layer.fit(X_train, y_train)

y_pred = perceptron_layer.predict(X_test)
```

Warstwa perceptronów została nauczona na zestawie punktów do trenowania. Następnie, na danych testowych, które model widzi po raz pierwszy, zastosowaliśmy go do klasyfikacji rożnych gatunków.

3.3 Wyniki

Do oceny jakości pracy modelu utworzono *confusion matrix*:



Rysunek 17: Confusion Matrix: Klasyfikacja gatunków irysa

Zatem obliczono *Accuracy*:

```
Accuracy: 63.33%
```

3.3.1 Wnioski

Na podstawie otrzymanej macierzy pomyłek (Rysunek 17) oraz obliczonej dokładności (ang. *accuracy*) na poziomie 63.33%, można wyciągnąć następujące wnioski

dotyczące działania algorytmu:

- **Idealna klasyfikacja klasy 0:** Model bezbłędnie rozpoznał wszystkie 10 próbek należących do klasy 0 (najprawdopodobniej *Iris setosa*). Potwierdza to fakt, że dane dla tego gatunku są liniowo separowalne od pozostałych, co pozwala perceptronowi na łatwe wyznaczenie poprawnej granicy decyzyjnej.
- **Problem z rozróżnieniem klas 1 i 2:** Algorytm poprawnie sklasyfikował wszystkie 9 próbek z klasy 1, jednak całkowicie zawiódł w przypadku klasy 2. Wszystkie 11 próbek rzeczywistej klasy 2 zostało błędnie zaklasyfikowanych jako klasa 1.
- **Praktyczne potwierdzenie ograniczeń modelu:** Przedstawione wyniki stanowią wyraźny dowód na teoretyczne ograniczenia jednowarstwowej sieci perceptronów. Brak separowalności liniowej między gatunkami *Iris versicolor* i *Iris virginica* uniemożliwił algorytmowi znalezienie optymalnej płaszczyzny podziału. W efekcie model zoptymalizował wagi tak, że scalił obie nakładające się na siebie klasy w jedną kategorię.

Matematyczny model klasycznego perceptronu to liniowy klasyfikator binarny, którego funkcja aktywacji wygląda następująco:

$$f(\mathbf{x}) = sign(\mathbf{w}^T\mathbf{x} + b)$$

Gdzie $\mathbf{w}$ to wektor wag, $\mathbf{x}$ to wektor cech, a b to obciążenie (ang. *bias*). Równanie to opisuje hiperpłaszczyznę, która dzieli przestrzeń cech na dwie części za pomocą linii prostej (lub płaskiej powierzchni w przestrzeni wielowymiarowej).

Perceptron gwarantuje zbieżność (znalezienie rozwiązania) tylko w przypadku, gdy klasy są separowalne liniowo — to znaczy, gdy można między nimi poprowadzić absolutnie prostą granicę bez nakładania się na siebie.

W zbiorze danych Iris:

- *Iris setosa* znajduje się daleko od dwóch pozostałych klas w przestrzeni cech. Jest on separowalny liniowo, dlatego perceptron z łatwością go izoluje.
- *Iris versicolor* oraz *Iris virginica* częściowo się pokrywają ze względu na swoje parametry fizyczne (długość i szerokość płatków oraz działek kielicha). Przeprowadzenie linii prostej, która idealnie oddzieliłaby jeden gatunek od drugiego, jest matematycznie niemożliwe. Nie są one separowalne liniowo.

Ponieważ algorytm próbuje znaleźć linię prostą tam, gdzie ona nie istnieje, wagi dla tych dwóch klas będą oscylować w nieskończoność (jeśli nie zostanie określone kryterium zatrzymania), a algorytm zawsze będzie popełniał błędy na przykładach granicznych.

4 Zadanie 3

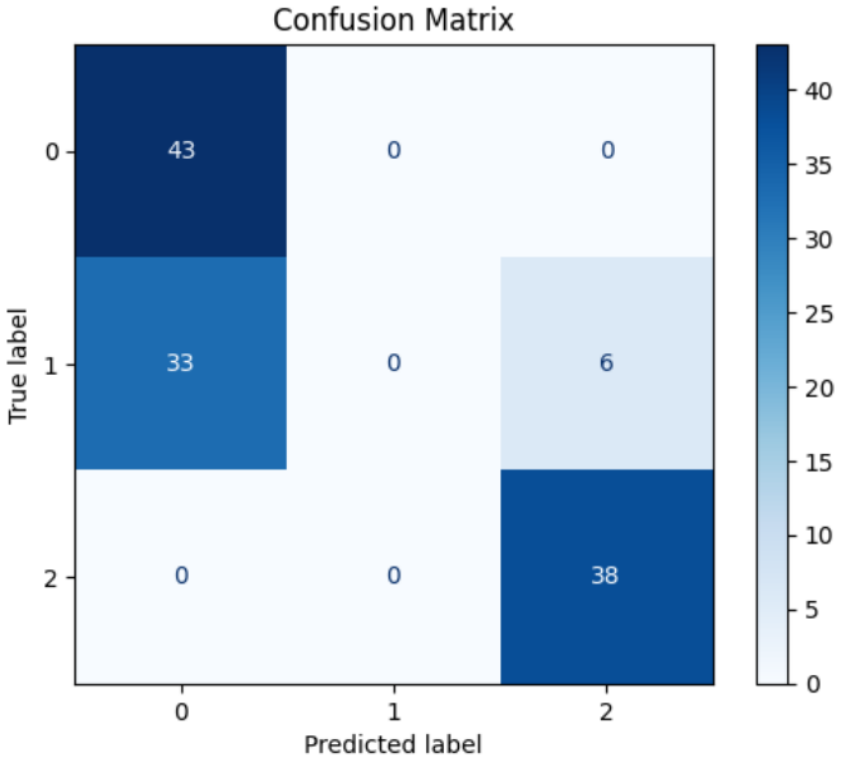
Celem trzeciego zadania jest zbadanie wpływu metody podziału zbioru danych na wyniki klasyfikacji. W ramach tego etapu zbiór irysów zostanie podzielony na dane uczące i testowe na co najmniej trzy różne sposoby.

4.1 80% - dane do testowania

Zobaczymy, co się stanie, jeśli model będzie uczony na znacznie mniejszym zbiorze.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.8,
↪  random_state=42)
```

Utworzono ponownie *Confusion Matrix*:



Rysunek 18: Confusion Matrix: Klasyfikacja gatunków irysa. 80% to dane do testowania

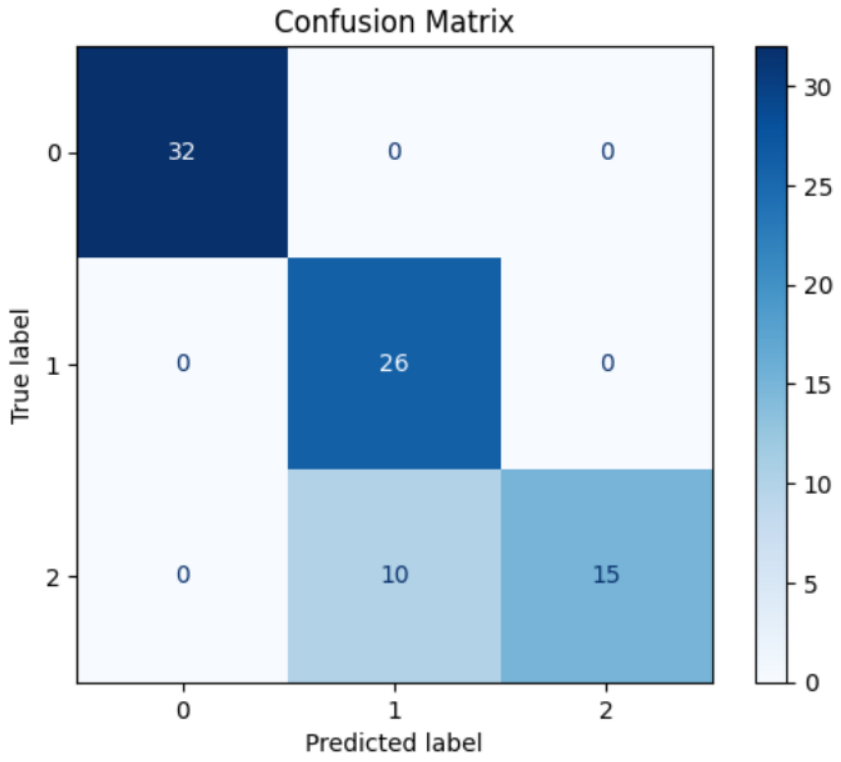
```
Accuracy: 67.50%
```

Zauważalnym i pozornie nielogicznym zjawiskiem jest fakt, że pomimo drastycznego zmniejszenia zbioru uczącego ogólna dokładność (ang. *Accuracy*) modelu wzrosła z 63.33% (dla 80% danych uczących) do 67.50% (dla 20% danych uczących). W modelu trenowanym na mniejszym zbiorze algorytm całkowicie utracił zdolność rozpoznawania klasy 1. Jednakże zbiór testowy był na tyle duży (120 próbek), że poprawne odgadnięcie wszystkich łatwiejszych do odseparowania próbek z klasy 0 (43 sztuki) oraz klasy 2 (38 sztuk) wystarczyło, aby matematyczny stosunek poprawnych odpowiedzi do wszystkich prób (81/120) okazał się wyższy. Dowodzi to, że model o wyższej dokładności ogólnej (67.50%) jest w rzeczywistości znacznie gorszym i mniej użytecznym klasyfikatorem, ponieważ uległ degeneracji do rozpoznawania tylko dwóch z trzech klas.

4.2 55% - dane do testowania

Wyniki dla zbioru, gdzie 55% punktów to dane uczące:

```
Accuracy: 87.95%
```



Rysunek 19: Confusion Matrix: Klasyfikacja gatunków irysa. 55% to dane do testowania

Eksperyment, w którym zbiór uczący stanowił 55% wszystkich danych, przyniósł najwyższą z zaobserwowanych dokładności (ang. *Accuracy*) na poziomie 87.95%. Jest to wynik zbliżony do teoretycznego maksimum możliwości jednowarstwowego perceptronu dla zbioru Iris.

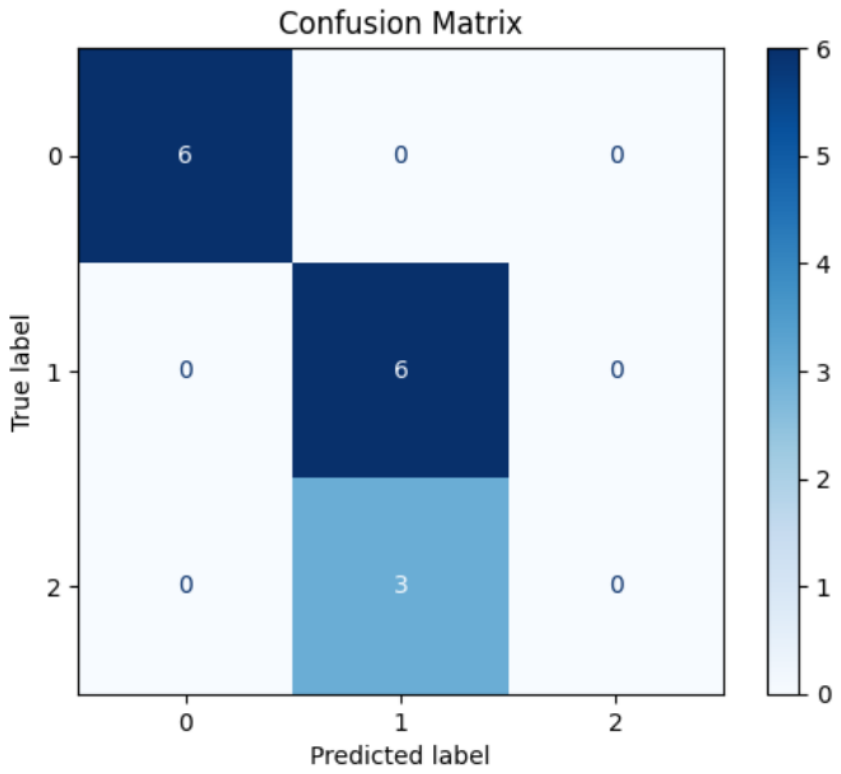
Z analizy macierzy pomyłek płyną następujące wnioski:

- **Wysoka skuteczność:** Model bezbłędnie sklasyfikował wszystkie próbki należące do klasy 0 (32 próbki) oraz klasy 1 (26 próbek). Oznacza to, że proporcja 55% danych uczących dostarczyła wystarczająco reprezentatywnych wzorców, aby model mógł wyznaczyć optymalną, użyteczną granicę decyzyjną.
- **Potwierdzenie braku separowalności liniowej:** Wszystkie 10 popełnionych błędów dotyczyło wyłącznie klasy 2, która została błędnie zinterpretowana jako klasa 1.
- **Ostateczna granica modelu:** Wynik ten jest klasycznym dowodem na to, że hiperpłaszczyzna (prosta linia) wyznaczona przez algorytm zawsze musi dokonać „kompromisu” w obszarze, gdzie gatunki *Iris versicolor* i *Iris virginica* się przenikają. W tym konkretnym przypadku optymalizacja wag sprawiła, że granica objęła całą klasę 1, wymuszając tym samym błędy na nakładających się punktach z klasy 2.

4.3 10% - dane do testowania

Wyniki dla zbioru, gdzie 10% punktów to dane uczące:

```
Accuracy: 80.00%
```



Rysunek 20: Confusion Matrix: Klasyfikacja gatunków irysa. 10% to dane do testowania

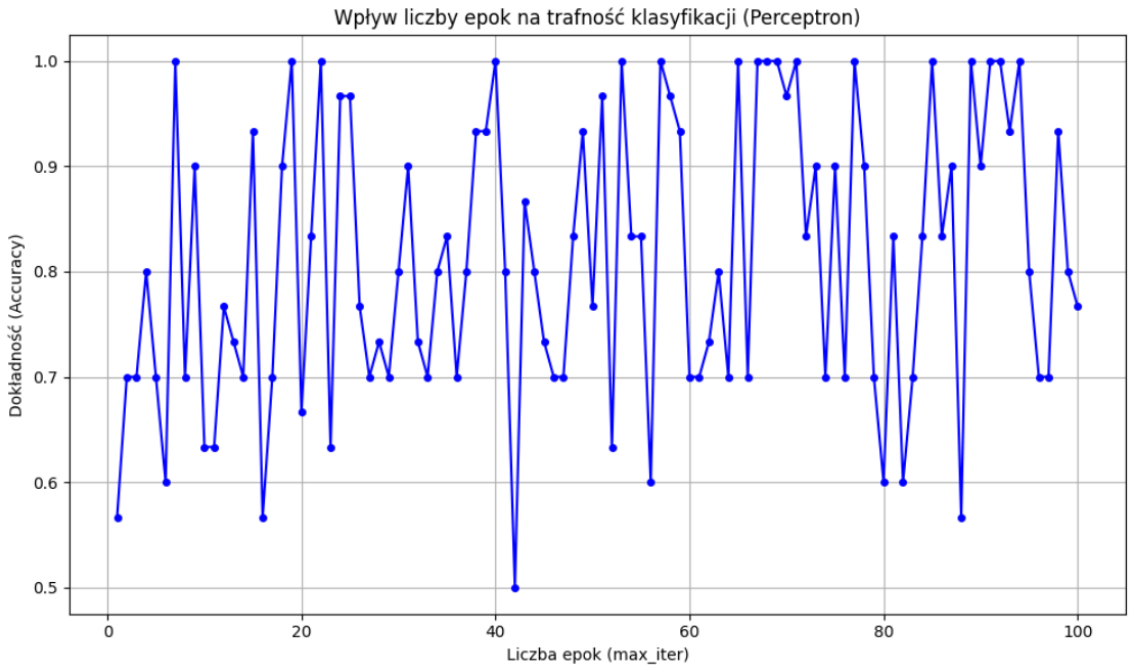
Zastosowanie bardzo małego zbioru testowego (10%, czyli 15 próbek) pozwoliło na uzyskanie dokładności na poziomie 80.00%. Klasyfikator bezbłędnie rozpoznał próbki z klas 0 i 1, jednak całkowicie pomylił się przy klasie 2, przypisując wszystkie jej 3 testowe przykłady do klasy 1. Stosunkowo wysoka wartość *Accuracy* jest w tym przypadku jedynie efektem bardzo małej próby statystycznej. Wynik ten po raz kolejny potwierdza, że model nie jest w stanie wyznaczyć prawidłowej granicy między nieliniowo separowalnymi klasami 1 i 2.

5 Zadanie 4

Celem tego zadania jest zbadanie wpływu liczby epok uczących na trafność klasyfikacji zbioru Iris. Wymaga to trenowania modelu dla różnych wartości iteracji (począwszy od bardzo małych, np. 1-2 epok) przy jednoczesnym wyłączeniu mechanizmów wczesnego zatrzymywania uczenia (m.in. poprzez modyfikację parametru `tol`).

```
epochs_list = list(range(1, 101))
accuracies = []

for epochs in epochs_list:
    clf = Perceptron(max_iter=epochs, tol=None, random_state=42)
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    accuracies.append(accuracy_score(y_test, y_pred))
```

Wygenerowano wykres zależności *Accuracy* od liczby epok:



Rysunek 21: Wpływ liczby epok na trafność klasyfikacji

5.1 Wnioski

- **Niska trafność w początkowej fazie:** Przy bardzo małej liczbie epok (np. 1–5), dokładność modelu jest bardzo niestabilna. Wagi perceptronu nie zdążyły się jeszcze odpowiednio dostosować, co skutkuje przypadkowymi i często błędnymi predykcjami.
- **Efekt oscylacji (braku zbieżności):** Wraz ze wzrostem liczby iteracji, dokładność nie rośnie monotonicznie do 100%, lecz nieustannie oscyluje. Zjawisko to jest bezpośrednią konsekwencją braku separowalności liniowej między gatunkami *Iris versicolor* i *Iris virginica*. Algorytm w każdej kolejnej epoce natrafia na błędnie sklasyfikowane punkty graniczne i nieustannie modyfikuje wagi, przesuwając hiperpłaszczyznę podziału w obie strony.

6 Spis Literatury i odpowiedne linki

- Perceptron – Wikipedia, wolna encyklopedia
  https://pl.wikipedia.org/wiki/Perceptron
- Linear separability - Wikipedia, wolna encyklopedia
  https://en.wikipedia.org/wiki/Linear_separability